

Why Bias is reduced with weighted average (AdaBoost)

AdaBoost labels take values of -1 or 1
 weight of a sample to be chosen changes

1. Initialize weights for samples 1. $w_i^{(1)} = \frac{1}{N}$ ($i = 1 - N$)
 $N = \#$ of samples

2. Repeat steps 3-5 for m -th weak learner ($m = 1 - M$)

3. Train the weak learner to minimize:

$$(\text{error func}) E^{(m)} = \frac{\sum_i w_i^{(m)} [\hat{y}_i^{(m)} \neq y_i]}{\sum_i w_i^{(m)}}$$

4. Determine the weight for the trained learner

$$\alpha^{(m)} = \ln(1 - E^{(m)} / E^{(m)}) \rightarrow \text{the amount of "say"}$$

5. Update the weights for training data $w_i^{(m+1)} = \dots$

→ Back to 3
 with $(m+1)$ -th learner

- Implication

1. if $E^{(m)} < 0.5$, then $\frac{1 - E^{(m)}}{E^{(m)}} > 1$, resulting $\alpha^{(m)} > 0$

better learners obtain higher positive voting weight

2. Given $y_i \in \{-1, 1\}$ and $\hat{y}_i^{(m)} \in \{-1, 1\}$ $\begin{cases} y_i = 1 \text{ and } \hat{y}_i^{(m)}(x_i) = 1 & \text{or} \\ y_i = -1 \text{ and } \hat{y}_i^{(m)}(x_i) = -1 & \end{cases}$

Case A: $y_i = \hat{y}_i^{(m)}(x_i) \rightarrow \hat{y}_i^{(m)}(x_i) y_i = 1$

(correct case) then $w_i^{(m+1)} = w_i^{(m)} \exp(-\alpha_m)$

(Low Bias) since $\alpha_m > 0$, $\exp(-\alpha_m) < 1$ weight decreases

Case B: $\hat{y}_i^{(m)}(x_i) \neq y_i \rightarrow \hat{y}_i^{(m)}(x_i) y_i = -1$ $\begin{cases} y_i = 1 \text{ and } \hat{y}_i^{(m)}(x_i) = -1 \\ \text{or opposite} \end{cases}$

(Highly biased) then $w_i^{(m+1)} = w_i^{(m)} \exp(+\alpha_m)$

since $\alpha_m > 0$, $\exp(+\alpha_m) > 1$ weight increases

⇒ each learner targets the localized bias of the previous ensemble sequentially

* Other General Boosting Doesn't use weighted average

More Boosting

General procedure

Input: Sample Dist: D
 Base learning algo: L
 # of learning rounds: T

1. $D_1 = D$
2. for $t = 1 \dots T$:
3. $h_t = L(D_t)$; train a learner
4. $\epsilon_t = P_{x \sim D_t}(h_t(x) \neq f(x))$; evaluate the error
5. $D_{t+1} = \text{Adjust_Distribution}(D_t, \epsilon_t)$
6. end

Output: $H(x) = \text{Combine_Outputs}(\{h_1(x), \dots, h_T(x)\})$

AdaBoost: Additive model, gradient descent on an Exponential loss function (only for classification) $y \in \{-1, 1\}$

5. if $\epsilon_t > 0.5$ then break — odds ratio
6. $\alpha_t = \frac{1}{2} \ln(1 - \epsilon_t / \epsilon_t)$ — odds ratio
7. $D_{t+1}(x) = D_t(x) / Z_t \times \begin{cases} \exp(-\alpha_t) & \text{if } h_t = f \\ \exp(\alpha_t) & \text{if } h_t \neq f \end{cases}$
 $= \frac{D_t(x) \exp(-\alpha_t \cdot f(x) \cdot h_t(x))}{Z_t}$

8. end

Output: $\text{sign}(H(x))$ with $H(x) = \sum_{t=1}^T \alpha_t h_t(x)$ ← additive model

$H(x)$ = continuous real number, $f(x) \cdot H(x)$ = margin (physical distance of point x to the model's decision boundary)

sign of margin = correct / incorrect
 value of |margin| = confidence (high / low)

$$(e^u)' = e^u \cdot u'$$

No.

Date

Now, exponential loss $\ell_{\text{exp}}(v) = e^{-v}$

then the expected exponential loss over a data distribution is:

$$\ell_{\text{exp}}(H|D) = E_{x \sim D} [e^{-f(x)H(x)}]$$

$$\frac{\partial \ell_{\text{exp}}(H|D)}{\partial H(x)} = -e^{-H(x)} P(f(x)=1|x) + e^{H(x)} P(f(x)=-1|x)$$

If $H(x)$ can minimize the loss, it can solve by $\frac{\partial \ell_{\text{exp}}(H|D)}{\partial H(x)} = 0$

$$\text{i.e., } H(x) = \frac{1}{2} \ln \frac{P(f(x)=1|x)}{P(f(x)=-1|x)}$$

$$\text{then } \text{sign}(H(x)) = \begin{cases} 1, & P(f(x)=1|x) > P(f(x)=-1|x) \\ -1, & \text{otherwise} \end{cases}$$

$$= \underset{y \in \{-1, 1\}}{\text{argmax}} P(f(x)=y|x)$$

• Finding the optimal weight α_t

Once the classifier h_t is generated from D_t , its weight α_t is estimated by letting $\alpha_t h_t$ minimize the exponential loss.

$$\ell_{\text{exp}}(\alpha_t h_t | D_t) = e^{-\alpha_t} \cdot P(\text{correct}) + e^{\alpha_t} \cdot P(\text{incorrect})$$

$$= e^{-\alpha_t} (1 - \epsilon_t) + e^{\alpha_t} \cdot \epsilon_t \quad \text{weighted error rate under } D_t$$

Set derivative to 0 and get (line 6)

→ larger α when ϵ is small

at iteration t , with a sequence of base learners h_1 to h_{t-1} , and corresponding weights, $H_{t-1} = \sum_{i=1}^{t-1} \alpha_i \cdot h_i(x)$

and AdaBoost adjusts the sample distribution such that the base learner h_t can learn from mistakes of H_{t-1}

• Deriving the dist. update.

No.

Taylor expansion $e^u = 1 + u + u^2/2! + u^3/3! + \dots$

Date

don't need constant to get critical point

the ideal h_t which can correct all mistakes of H_{t-1} should minimize:

$$\text{lexp}(H_{t-1} + h_t | D) = \mathbb{E}_{x \sim D} [e^{-f(x)(H_{t-1}(x) + h_t(x))}]$$

$$\rightarrow h_t(x) = \underset{h}{\operatorname{argmin}} \text{lexp}(H_{t-1} + h | D)$$

$$\text{now } \mathbb{E}_{x \sim D} [e^{-f(x)(H_{t-1}(x) + h_t(x))}]$$

$$\stackrel{||}{=} e^{-f(x)H_{t-1}(x)} \cdot e^{-f(x)h_t(x)} \stackrel{||}{=} (1 + u + u^2/2! + u^3/3! + \dots)$$

$$\text{since } f^2(x) = h_t^2(x) = 1, (-f(x)h_t(x))^2 = f^2(x)h_t^2(x) = 1$$

dropping $u^3/3!$ term and after approximates constant $\rightarrow$ dropped

$$\simeq \mathbb{E}_{x \sim D} [e^{-f(x)H_{t-1}(x)} \cdot (1 - f(x)h_t(x) + \frac{1}{2})]$$

$$\text{the } h_t(x) = \underset{h}{\operatorname{argmax}} \mathbb{E}_{x \sim D} [e^{-f(x)H_{t-1}(x)} \cdot f(x)h_t(x)]$$

now by definition weighted expectation means:

$$\mathbb{E}_{x \sim D} [w(x) \cdot g(x)] = \sum_x D(x) \cdot w(x) \cdot g(x)$$

$$\text{when } D_t(x) = D(x) \cdot w(x) / Z,$$

what it did:

move the w out of $\mathbb{E}$
into dist.

$$= Z \sum_x D_t(x) \cdot g(x) \quad (0 \sim 1)$$

$$= Z \cdot \mathbb{E}_{x \sim D_t} [g(x)]$$

now apply $w(x) = e^{-f(x)H_{t-1}(x)}$ and $g(x) = f(x)h_t(x)$

$$\text{and } \mathbb{E}_{x \sim D} [e^{-f(x)H_{t-1}(x)} \cdot f(x)h_t(x)] = Z \cdot \mathbb{E}_{x \sim D_t} [f(x)h_t(x)]$$

$$\text{where } D_t(x) = \frac{D(x) \cdot e^{-f(x)H_{t-1}(x)}}{Z}, \quad Z = \sum_x D(x) \cdot e^{-f(x)H_{t-1}(x)}$$

Since Z is just a positive constant $\rightarrow$ dropped

and
$$h_t(\underline{x}) = \arg \max_h \mathbb{E}_{\underline{x} \sim D_t} [f(\underline{x}) h(\underline{x})]$$

here,

H_{t-1} correct/confident	$f(\underline{x}) H_{t-1}(\underline{x})$ large pos.	$e^{(-f \cdot H_{t-1})}$ small	weight low
uncertain/new zero	near zero	≈ 1	medium
incorrect/confident	large neg.	large	high

since $f(\underline{x}) h_t(\underline{x}) = 1 - 2 \cdot \mathbb{I}[f(\underline{x}) \neq h_t(\underline{x})]$

$$h_t(\underline{x}) = \arg \min_h \mathbb{E}_{\underline{x} \sim D_t} [\mathbb{I}(f(\underline{x}) \neq h(\underline{x}))]$$

• next iteration prep (recursive form)

$$D_{t+1}(\underline{x}) \propto D_t(\underline{x}) \cdot e^{(-f(\underline{x}) \cdot d_t \cdot h_t(\underline{x}))}$$

which expands to the relationship between D_{t+1}, D_t (residue minimization)

$$D_{t+1}(\underline{x}) = \frac{D_t(\underline{x}) e^{(-f(\underline{x}) H_t(\underline{x}))}}{\mathbb{E}_{\underline{x} \sim D} [e^{(-f(\underline{x}) H_t(\underline{x}))}]}$$

$$D_t(\underline{x}) e^{(-f(\underline{x}) H_{t-1}(\underline{x}))} \cdot e^{(-f(\underline{x}) d_t h_t(\underline{x}))}$$

$$= D_t(\underline{x}) \cdot e^{(-f(\underline{x}) d_t h_t(\underline{x}))} \cdot \frac{\mathbb{E}_{\underline{x} \sim D} [e^{(-f(\underline{x}) H_{t-1}(\underline{x}))}]}{\mathbb{E}_{\underline{x} \sim D} [e^{(-f(\underline{x}) H_t(\underline{x}))}]}$$

$$Z_t = \sum_{\underline{x}} D_t(\underline{x}) \cdot e^{(-f(\underline{x}) d_t h_t(\underline{x}))}$$

h_t	$f(\underline{x}) h_t(\underline{x})$	$e^{(-f(\underline{x}) d_t h_t(\underline{x}))}$
correct	+1	$e^{(-d_t)}$
incorrect	-1	$e^{(d_t)}$

$$\Rightarrow D_{t+1}(\underline{x}) = \frac{D_t(\underline{x})}{Z_t} \times \begin{cases} e^{-d_t} & \text{if } h_t(\underline{x}) = f(\underline{x}) \\ e^{d_t} & \text{if } \neq \end{cases} \quad (\text{line 7})$$